

- **Explicación del algoritmo implementado:** se utilizó la subrutina `utils_read_column_to_stack` proveniente de la librería `Utils.s` que realiza un recorrido por todas las líneas del archivo `lecturas.csv` y colocando en el stack únicamente los datos de la columna que se desea analizar. Una vez con estos datos ya en el stack procedemos a realizar los cálculos mediante las subrutinas `med`, `var`, y `sds`:
 - `med`: la subrutina `med` se encarga de sumar todos los datos que están en el stack que formo `utils` y declarando datos como un acumulador, un contador con la cantidad completa de datos leídos y todos estos datos los utilizaremos para un loop, esto se logra mediante un loop de suma en el cual vamos tomando el contenido del stack al que se apunta durante ese momento y haciendo un corrimiento de 16 bytes para llegar al siguiente dato este dato es colocado en un acumulador previamente declarado vacío, luego restamos una unidad al contador que contiene el total de datos para saber donde termina el loop, se hace la comparación entre el contador y el valor de 0 si el contador no es mayor que 0 el loop termina de lo contrario se sigue el loop, si se termina el loop tomamos el valor acumulado de la suma y lo dividimos sobre la cantidad total de números analizados colocando el resultado en el registro `x22`.
 - `Var`: esta subrutina es muy parecida a la de la media, con ciertas diferencias en lugar de sumar directamente a un acumulador primero se realiza una resta entre el valor actual leído y la media previamente calculada, luego este resultado se multiplica por si mismo y posteriormente se hace la suma al acumulador, luego se resta una unidad al contador de descenso para saber donde detener el loop, se hace la comparación de control de loop, si es tiempo de detenerse se salta a división del acumulador y el numero total de datos leídos previamente, este es el resultado es guardado en el registro `x23`.
 - `Sdvs`: esta es una subrutina que realiza el calculo de la raíz cuadrada de la varianza previamente calculada, esto lo logramos mediante el método de newton y uno de sus casos especiales que se muestra en la imagen adjunta, esto lo logramos mediante el resultado de la varianza que se acaba de calcular y creando 2 copias de esta que se utilizaran como el valor del cual se quiere encontrar la raíz cuadrada y otro que se tomará como la primera aproximación que es exigida por el método, esto lo logramos primero realizando una división entre el valor de la primera aproximación y el numero del cual se quiere calcular la raíz, posteriormente a este resultado se le vuelve a sumar la aproximación inicial, para por ultimo hacer una división entre 2, con esto tenemos la nueva aproximación para el método, hacemos una comparación $\Rightarrow$ entre la aproximación actual y la aproximación anterior, si esta comparación es correcta significa que hemos llegado la raíz que buscamos y mediante el salto a una nueva subrutina llamada `done` donde guardamos el resultado de la raíz en el registro `x24` y culminamos la subrutina, por que hacemos esto, la razón es que si esta comparación es correcta significa que la aproximación actual no disminuyo drásticamente su valor con respecto a la aproximación anterior por ende podemos asumir que ya nos acercamos al valor de la raíz real, esto debido a que como se debe recordar estamos únicamente aproximando nuestros cálculos al valor entero mas cercano, si la comparación no es correcta el loop se vuelve a realizarse cambiando el valor de la aproximación actual por el valor calculado anteriormente.
 - Esta es la formula utilizada para calcular la raíz cuadrada mediante el método de newton.

$$x_{n+1} = \frac{1}{2} \left[x_n + \frac{m}{x_n} \right]$$

- **Escritura de la salida:**
 - Para esto utilizaremos las etiquetas en formato asciz que declaramos en la sección de .data, y generando un substack para la escritura actual, para esto también utilizaremos otra subrutina llamada copiar_str. La primera etiqueta no tenemos mas que colocarla en el stack debido a que ya tiene el formato requerido. Para las demás etiquetas debemos utilizaremos la subrutina copiar_str para cargar el la etiqueta y luego escribir_num la cual toma el valor del dato que estamos escribiendo ya sea media, varianza o desviación y se lo manda a escribir_num.
 - escribir_num: se crea otro substack y se invoca utils_itoa. Que transforma el formato numérico que le paso a un formato ascii con un carácter de control para saber el final del numero que escribimos.
- **Explicacion de los registros utilizados:** Principalmente se utilizaron registros de propósito general, registros de 32 bits cuando se necesitaba leer un único carácter. Y por ultimo registros Callee-saved, que guardan los resultados mientras están en la rutina y luego lo regresan a sus valores originales antes de hacer el retorno.
- **Explicación de los ciclos, saltos y subrutinas:** las subrutinas ya fueron abordadas anteriormente, así que hablaremos de los ciclos utilizados y las subrutinas, conectamos a las subrutinas mediante bl una instrucción la cual guarda la posición a la cual regresar una vez la subrutina sea completada, los bucles utilizados al menos en las primeras dos subrutinas son bucles simples con una condición que nos indica cuando detenernos esta es manejada mediante un contador descendente el cual se va reduciendo una unidad cada iteración, cuando este contador llega a 0 el ciclo termina y se guardan los datos en el registro de salida correspondiente. El bucle de la tercera subrutina es muy similar con la única diferencia de que no contamos con un contador que vaya disminuyendo en cada iteración, entonces controlamos el flujo del bucle mediante una comparación entre el resultado anterior del mismo bucle, el cual definimos nosotros para la primera iteración, y luego este mismo resultado se va actualizando por cada repetición del bucle hasta encontrar una iteración que logre